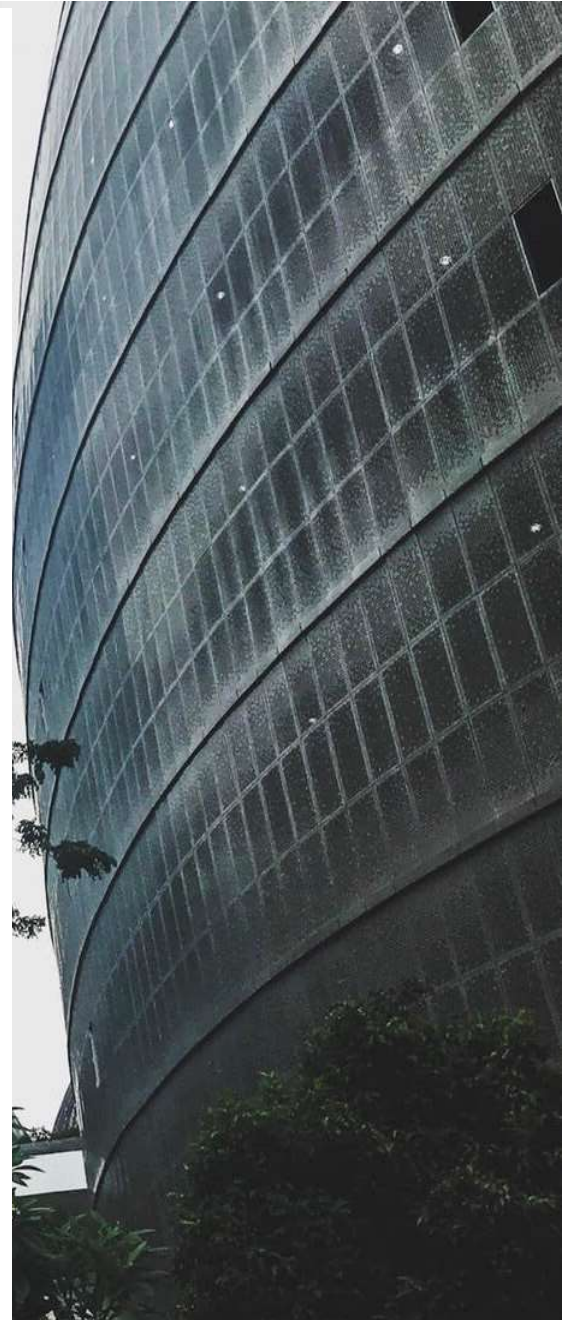


COURSE MATERIAL

**IF670 – Mobile Cross Platform Programming
S1 INFORMATICS GRADUATE PROGRAM
FACULTY OF ENGINEERING AND INFORMATICS**

**INFORMATICS STUDY PROGRAM
FACULTY OF ENGINEERING AND INFORMATION
UNIVERSITAS MULTIMEDIA NUSANTARA**

Gedung B Lantai 5, Kampus UMN
Jl. Scientia Boulevard, Gading Serpong, Tangerang-15811 Indonesia
Telp: +62-21.5422.0808 (ext. 1803), email: lab.fti@umn.ac.id, web: umn.ac.id



MODUL 11

(Supabase)

THEME DESCRIPTION

Students will learn how to integrate Backend as a Service (BaaS) using Supabase into a React Native Expo application. In this module, students will create a simple React Native Expo project, connect the application to Supabase, create a simple database table, and insert data from the mobile application into the Supabase table.

Supabase is used as a backend service that provides a PostgreSQL database, automated APIs, authentication, storage, and other backend features. In this practical session, the main focus is on using the Supabase database to store simple data from the React Native Expo application.

WEEKLY LEARNING GOAL (SUB-GOAL LEARNING)

Upon completing this practical session, students are expected to be able to:

1. Understand the role of Supabase as a Backend as a Service.
2. Create and configure a Supabase project online.
3. Create simple tables in Supabase.
4. Install and configure Supabase within a React Native Expo project.
5. Perform simple data inserts from the React Native Expo application into Supabase.
6. Implement Supabase integration into the project continued from the previous week.

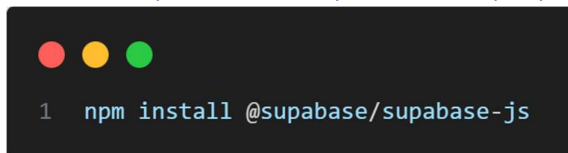
SOFTWARE / HARDWARE REQUIREMENTS

1. Laptop / PC
2. Android Studio / Xcode or a mobile phone with Expo Go
3. A code editor such as Visual Studio Code
4. Node.js and npm
5. An Expo project
6. A Supabase account
7. An internet connection

PRACTICUM STEPS

1. **Create a new Expo project.**
2. **Install the Supabase JavaScript Client.**

Install the Supabase JavaScript client library in your Expo project:



```
1  npm install @supabase/supabase-js
```

The @supabase/supabase-js dependency is used so that the React Native app can communicate with the Supabase database.

3. **Create a Supabase Project.**

Open the Supabase Dashboard website, then follow these steps:

1. Log in using your Supabase account.
2. On the Supabase dashboard, click the New Project button.

Create a new project
Your project will have its own dedicated instance and full Postgres database.
An API will be set up so you can easily interact with your new database.

Organization Achxter FREE

Project name Project name

Database password Type in a strong password

This is the password to your Postgres database, so it must be strong and hard to guess. [Generate a password.](#)

Region Asia-Pacific

Select the region closest to your users for the best performance.

Security

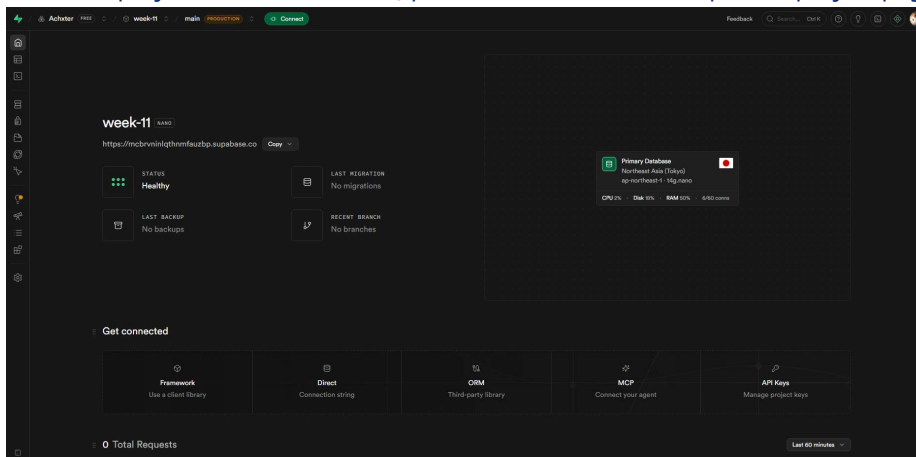
- ☒ **Enable Data API**
Autogenerate a RESTful API for your public schema.
Recommended if using a client library like [supabase-js](#).
- ☒ **Automatically expose new tables and functions**
Grants privileges to Data API roles by default, exposing new tables and functions.
We recommend disabling this to control access manually.
- ☐ **Enable automatic RLS**
Create an event trigger that automatically enables Row Level Security on all new tables in the public schema.

ADVANCED CONFIGURATION >

Cancel Create new project

3. Select an available organisation.
4. Enter a project name, e.g. week11-supabase.
5. Create a database password.
6. Select the nearest server region, e.g. Asia-Pacific.
7. Click Create new project.
8. Wait until the project creation process is complete.

Once the project has been created, you will be taken to the Supabase project page.



The screenshot shows the Supabase project page for a project named 'week-11'. The page includes a sidebar with navigation icons, a main content area with project details, and a 'Get connected' section at the bottom.

Project Details:

- Project Name:** week-11
- URL:** <https://mcbrvniqlthrmfauzbp.supabase.co>
- STATUS:** Healthy
- LAST MIGRATION:** No migrations
- LAST BACKUP:** No backups
- RECENT BRANCH:** No branches

Primary Database:

- Region: (Tokyo)
- Size: 10GB
- Storage: 10GB
- Connections: 6/10

Get connected:

- Framework:** Use a client library
- Direct:** Connection string
- ORM:** Third-party library
- MCP:** Connect your agent
- API Keys:** Manage project keys

0 Total Requests (Last 60 minutes)

4. Creating the users table.

Go to the Table Editor menu, then click Create a table. Populate the table with the following structure:

☒ **Enable Row Level Security (RLS)** RECOMMENDED
Restrict access to your table by enabling RLS and writing Postgres policies.

Policies are required to query data
You need to create an access policy before you can query data from this table. Without a policy, querying this table will return an **empty array** of results. You can create policies after saving this table.

[Documentation](#)

☐ **Enable Realtime**
Broadcast changes on this table to authorized subscribers.

Columns [About data types](#) [Import data from CSV](#)

Name	Type	Default Value	Primary
id	# int8	NULL	☒
created_at	timestampz	now()	☐
first	T text	NULL	☐
last	T text	NULL	☐
birth	# int8	NULL	☐

5. Creating a Supabase Policy.

Supabase uses Row Level Security (RLS) to manage data access. If RLS is enabled but no policy has been created, the mobile app will be unable to insert or read data.

For the purposes of this basic practical exercise, create a policy to allow the application to perform all data operations on the users table:

Click the 'Add RLS policy' button.

users +

Filter by id, created_at, first... or ask AI

Sort [Add RLS policy](#) Role postg

RLS is enabled for this table, but no policies are set. Select queries may return 0 results.

Click the 'Create policy' button to begin creating the policy.

Auto-enable RLS for new tables
Create an event trigger that enables Row Level Security on all new tables. [Learn more](#)

schema: public users

users [Disable RLS](#) [Create policy](#)

No data will be returned via the Data API as no RLS policies exist on this table.

No policies created yet

Enter a name for the policy you wish to create.

Select the Policy Command 'ALL' to grant full access to all operations.

Click the 'Save policy' button to save the policy you have created.

Create a new Row Level Security policy

Policy Name

Enable all

Table on clause

public.users

Policy Behavior as clause

Permissive

Policy Command for clause

☐ SELECT
 ☐ INSERT
 ☐ UPDATE
 ☐ DELETE
 ☒ ALL

Target Roles to clause

Defaults to all (public) roles if none selected

USE OPTIONS ABOVE TO EDIT

```

1 create policy "Enable all"
2 on "public"."users"
3 as PERMISSIVE
4 for ALL
5 to public
6 using (
7   | true
8 ) with check (
9   -- Provide a SQL expression for the with check statement
10  );

```

☒ Use check expression

Cancel

Save policy

Templates

Search templates

SELECT

Enable read access for all users

This policy gives read access to your table for all users via the SELECT operation.

INSERT

Enable insert for authenticated users only

This policy gives Insert access to your table for all authenticated users only.

DELETE

Enable delete for users based on user_id

This policy assumes that your table has a column "user_id", and allows users to delete rows which the "user_id" column matches their ID

INSERT

Enable insert for users based on user_id

This policy assumes that your table has a column "user_id", and allows users to insert rows which the "user_id" column matches their ID

UPDATE

Policy with table joins

Query across tables to build more advanced RLS rules
Assuming 2 tables called teams and members, you can query both tables in the policy to control access to the members table.

ALL

Policy with security definer functions

Useful in a many-to-many relationship where you want to restrict access to the linking

Note: The policy above is intended for basic practice only. In production applications, database access must be restricted using more secure authentication and policies.

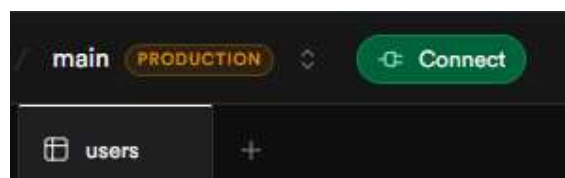
6. Retrieving the Supabase Project URL and API Key.

To connect the application to Supabase, students will need the following two pieces of information:

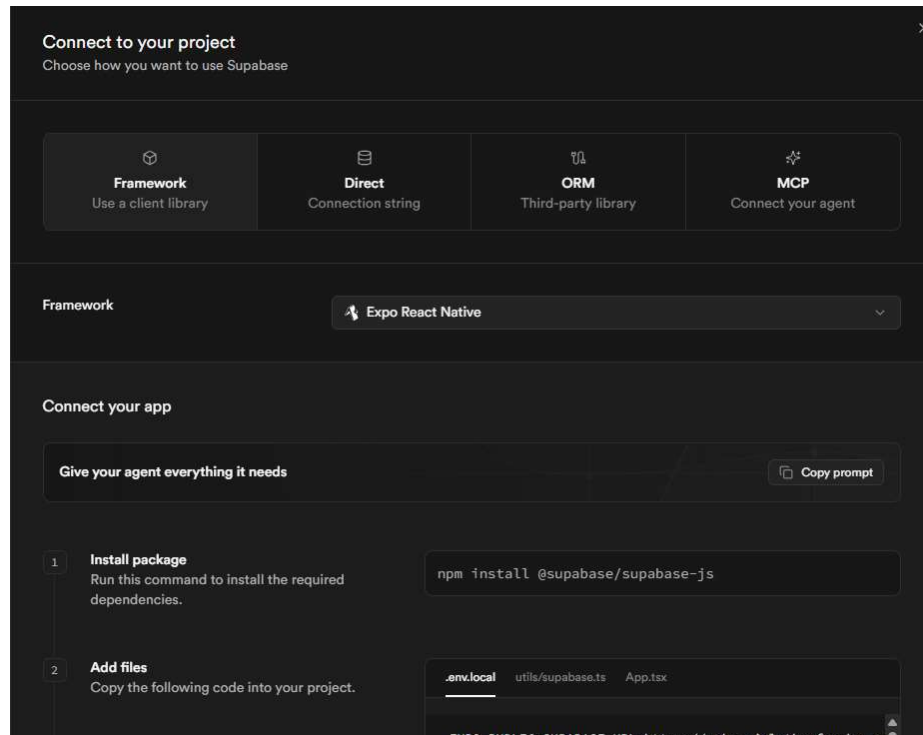
- Supabase Project URL
- Supabase anonymous public API key or publishable key

Steps to retrieve them:

- Click the Connect button



- Select the Expo React Native framework



3. Copy the public URL and public key provided by Supabase on that page.

7. Creating an Environment Variable File

In the root folder of the Expo project, create a new file named `.env`. Populate the `.env` file as follows:

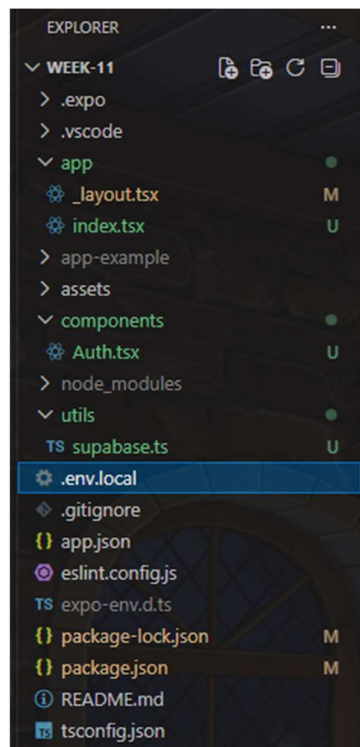
```
1 EXPO_PUBLIC_SUPABASE_URL=https://your-project-id.supabase.co
2 EXPO_PUBLIC_SUPABASE_ANON_KEY=your-anon-or-publishable-key
```

1. Replace the value `'https://your-project-id.supabase.co'` with your own public URL.
2. Replace the value `'your-anon-or-publishable-key'` with your own Supabase public key.

8. Creating the Supabase Configuration File.

1. Create a new folder named `'lib'` in the project root.
2. Inside that folder, create a new file named `'supabase.ts'`.

The folder structure will be as follows:



1. Insert the 'lib/supabase.ts' file with the following code:

```
1 import { createClient } from '@supabase/supabase-js'
2
3 const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL as string
4 const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY as string
5
6 export const supabase = createClient(supabaseUrl, supabaseAnonKey)
```

9. Testing the Project Connection with Supabase

1. Modify the index.tsx file to match the following code:


```

1  import React from 'react'
2  import { Alert, Button, StyleSheet, Text, View } from 'react-native'
3  import { supabase } from '../utils/supabase'
4
5  const Index = () => {
6    const addData = async () => {
7      try {
8        const { error } = await supabase.from('users').insert([
9          {
10             first: 'Ada',
11             last: 'Lovelace',
12             birth: 1815,
13           },
14         ])
15
16         if (error) throw error
17
18         Alert.alert('Success', 'Data added successfully.')
19       } catch (error: any) {
20         Alert.alert('Error', error?.message ?? 'Failed to add data.')
21       }
22     }
23
24     return (
25       <View style={styles.container}>
26         <Text style={styles.title}>Supabase connection test</Text>
27         <Button title="Add Data" onPress={addData} />
28       </View>
29     )
30   }
31
32   export default Index
33
34   const styles = StyleSheet.create({
35     container: {
36       flex: 1,
37       justifyContent: 'center',
38       alignItems: 'center',
39       padding: 24,
40       backgroundColor: '#f5f7fb',
41     },
42     title: {
43       fontSize: 20,
44       fontWeight: '600',
45       marginBottom: 16,
46       color: '#1f2937',
47     },
48   })

```

2. Then open the application via the emulator or Expo Go. Next, press the Add Data button.
3. If successful, the application will display a message stating that the data has been successfully added. The newly entered data will also appear in the users table.

ASSIGNMENT

Modify the assignments from weeks 9 and 10 by adding Supabase integration. Save your camera captures to Supabase and include the geolocation where the photo was taken. Here is an example of data that has been successfully uploaded to Supabase.

photo

+

Filter by id, created_at, latitude... or ask AI

Sort

RLS policy

Role postgres

Index Advisor

Insert

	id int8	created_at timestampz	latitude text	longitude text	image_url text
	2	2026-05-03 17:09:15.332935+0	-6.1482815	106.7127028	https://mcbrvnlnlqthnmfauzbp.supabase.

Images camera

camera

photo-1777454452152.jpeg

photo-177745450416.jpeg

photo-1777828062446.jpeg

photo-1777828153641.jpeg

photo-1777828153641.jpeg

image/jpeg - 82.98 KB

Added on

5/4/2026, 12:09:14 AM

Last modified

5/4/2026, 12:09:14 AM

Download

Get URL

Delete file

REFERENSI